3/9/26 DAY 11:

Constructor Execution with Parameters(Inheritance)

- When a derived class constructor is called, the base class constructor is executed first, followed by the derived class constructor.
- If the base class constructor has parameters, the derived class constructor must explicitly call the base class constructor with the appropriate arguments.

```
class Base {
public:
    Base(int x) {
        // Base class constructor
    }
};

class Derived : public Base {
public:
    Derived(int x, int y) : Base(x) {
        // Derived class constructor
    }
};
```

Function Overriding

- Function overriding occurs when a derived class provides a specific implementation of a function that is already defined in its base class.
- The function in the derived class must have the same name, return type, and parameters as the function in the base class.
- It allows the derived class to provide its own behavior for the function, while still maintaining the interface defined by the base class.
- It is important to use the `virtual` keyword in the base class function declaration to enable polymorphic behavior. This allows the derived class function to be called through a base class pointer or reference.

```
class Base {
public:
    virtual void display() {
        // Base class function
    }
};

class Derived : public Base {
public:
    void display() override {
        // Derived class function
```

```
    }
};
```

> If the virtual keyword is not used in the base class, the function in the derived class will hide the base class function instead of overriding it.

Overloading vs Overriding

- Overloading occurs when multiple functions have the same name but different parameter lists within the same scope. It is resolved at compile time based on the arguments passed to the function.
- Overriding occurs when a derived class provides a specific implementation of a function that is already defined in its base class. It is resolved at runtime based on the actual object type being referenced.

Overloading	**Overriding**
Occurs within the same class or scope	Occurs in a derived class
Resolved at compile time	Resolved at runtime
Function signature can be different	Function signature must be the same
Used to provide multiple implementations of a function with the same name	Used to provide a specific implementation of a function in a derived class

Polymorphism

- Polymorphism is a fundamental concept in object-oriented programming that allows objects of different classes to be treated as objects of a common base class.
- It enables a single interface to represent different underlying forms (data types).
- Polymorphism can be achieved through function overloading, operator overloading, and function overriding.

Types of Polymorphism

There are two main types of polymorphism in C++:

1. Compile-Time Polymorphism

- Compile-time polymorphism, also known as static polymorphism, is achieved through function overloading and operator overloading.
- It is resolved at compile time, meaning that the compiler determines which function or operator to invoke based on the arguments provided.
- Examples include function overloading and operator overloading.

```
class Math {
public:
    int add(int a, int b) {
        return a + b;
    }
```

```
    double add(double a, double b) {
        return a + b;
    }
};
```

2. Run-Time Polymorphism

- Run-time polymorphism, also known as dynamic polymorphism, is achieved through function overriding and virtual functions.
- It is resolved at runtime, meaning that the decision of which function to invoke is made based on the actual object type being referenced, rather than the type of the pointer or reference.
- Examples include function overriding and the use of virtual functions.

```
class Base {
public:
    virtual void display() {
        // Base class function
    }
};
class Derived : public Base {
public:
    void display() override {
        // Derived class function
    }
};
```

Virtual Functions

- Virtual functions are member functions in a base class that can be overridden in derived classes.
- They enable run-time polymorphism, allowing the program to determine which function to call based on the actual object type being referenced, rather than the type of the pointer or reference.
- To declare a virtual function, the **virtual** keyword is used in the base class function declaration.
- When a virtual function is called through a base class pointer or reference, the derived class's implementation of the function is invoked if it exists; otherwise, the base class's implementation is used.

```
int main() {
    Base* ptr = new Derived();
    ptr->display(); // Calls Derived::display()
    return 0;
}
```

Early Binding vs Late Binding

Early Binding

- Early binding, also known as static binding, occurs at compile time.

- The function to be called is determined by the compiler based on the type of the pointer or reference.

Late Binding

- Late binding, also known as dynamic binding, occurs at runtime.
- The function to be called is determined by the actual object type being referenced, allowing for polymorphic behavior.

Early Binding	**Late Binding**
Occurs at compile time	Occurs at runtime
Function to be called is determined by the compiler	Function to be called is determined by the actual object type at runtime
More efficient	Slight performance overhead
Used for non-virtual functions and function overloading	Used for virtual functions and function overriding
Less flexible	More flexible
Cannot be changed at runtime	Can be changed at runtime

Upcasting

- Upcasting is the process of converting a pointer or reference of a derived class to a pointer or reference of its base class.
- It allows a derived class object to be treated as an object of its base class, enabling polymorphic behavior.

```
class Base {
public:
    virtual void display() {
        // Base class function
    }
};
class Derived : public Base {
public:
    void display() override {
        // Derived class function
    }
};

int main() {
    Derived derivedObj;
    Base* basePtr = &derivedObj; // Upcasting
    basePtr->display(); // Calls Derived::display()
    return 0;
}
```

Downcasting

- Downcasting is the process of converting a pointer or reference of a base class to a pointer or reference of its derived class.
- It allows access to the derived class's specific members and functions, but it should be done with caution, as it can lead to undefined behavior if the object being downcasted is not actually of the derived class type.
- Downcasting can be performed using `static_cast` or `dynamic_cast`. The latter is safer, as it performs a runtime check to ensure the validity of the cast.

```
int main() {
    Base* basePtr = new Derived();
    // Downcasting using dynamic_cast

    Base* basePtr1 = new Derived();

    Derived* derivedPtr = dynamic_cast<Derived*>(basePtr);  // Safe downcasting
using dynamic_cast
    Derived* derivedPtr1 = (Derived*)(basePtr1);  // Unsafe downcasting using C-
style cast

    if (derivedPtr) {
        // Safe to use derivedPtr
    }
    return 0;
}
```

dynamic_cast

- **dynamic_cast** is a C++ operator used for safely downcasting pointers or references in an inheritance hierarchy.
- It performs a runtime check to ensure that the cast is valid, returning a null pointer (for pointers) or throwing a **std::bad_cast** exception (for references) if the cast is not valid.
- **dynamic_cast** is typically used in conjunction with polymorphic types (i.e., classes with at least one virtual function) to ensure safe type conversions.

```
class Animal {
public:
    virtual void voice() {
        // Base class function
    }
};
class Dog : public Animal {
public:
    void voice() override {
        // Derived class function
    }
};
```

```
class Cat : public Animal {
public:
    void voice() override {
        // Another derived class function
    }
};

int main() {
    Animal* basePtr = new Dog();
    Dog* dogPtr = dynamic_cast<Dog*>(basePtr); // Valid downcast
    if (dogPtr) {
        // Safe to use dogPtr
    }

    Animal* anotherBasePtr = new Cat();
    Cat* catPtr = dynamic_cast<Cat*>(anotherBasePtr); // Valid downcast
    if (catPtr) {
        // Safe to use catPtr
    }
    return 0;
}
```

Object Slicing

- Object slicing occurs when an object of a derived class is assigned to an object of its base class, resulting in the loss of the derived class's specific attributes and behaviors.
- This happens because the base class object can only hold the data members defined in the base class, effectively "slicing off" the derived class's additional members.

```
class Animal {
  int legs;
public:
    Animal(int l) : legs(l) {}
    virtual void voice() {
        // Base class function
    }
    void display() {
        cout << "Number of legs: " << legs << endl;
    }
};

class Dog : public Animal {
  string breed;
public:
    Dog(int l, string b) : Animal(l), breed(b) {}
    void voice() override {
        // Derived class function
    }
    void display() {
        cout << legs << " " << breed << endl; // This will cause an error because
'legs' is private in Animal
```

```
    }
};

int main() {
    Dog dog;
    Animal animal = dog; // This will cause object slicing

    animal.display(); // Only displays the base class attributes, derived class
attributes are lost
    return 0;
}
```

Virtual Destructor

- A virtual destructor is a destructor in a base class that is declared with the `virtual` keyword.
- It ensures that when a derived class object is deleted through a base class pointer, the derived class's destructor is called first, followed by the base class's destructor. This prevents resource leaks and ensures proper cleanup of derived class resources.
- It is important to always declare a virtual destructor in a base class if you plan to delete objects of derived classes through base class pointers.
- If there is a virtual function in the base class, it is a good practice to also declare a virtual destructor to ensure proper cleanup of derived class resources.

```
class Base {
public:
    virtual ~Base() {
        // Base class destructor
    }
};

class Derived : public Base {
public:
    ~Derived() {
        // Derived class destructor
    }
};
```

Diamond Problem

- It occurs in multiple inheritance when two classes inherit from the same base class, and a derived class inherits from both of these classes. This can lead to ambiguity and redundancy in the inheritance hierarchy.
- The diamond problem can be resolved using virtual inheritance, which ensures that only one instance of the base class is shared among the derived classes, preventing ambiguity and redundancy.

```
    A
   / \
```

```
   B   C
    \ /
     D
```

Virtual Base Class

- A virtual base class is a base class that is specified as virtual in the inheritance hierarchy.
- It is used to solve the diamond problem in multiple inheritance by ensuring that only one instance of the base class is shared among the derived classes.

```
class A {
public:
int a;
};
class B : virtual public A {
public:
int b;
};
class C : virtual public A {
public:
int c;
};
class D : public B, public C {
public:
int d;
};

int main() {
    D obj;
    obj.a = 10; // Accessing the single instance of A

    cout << "Value of a: " << obj.a << endl; // Output: Value of a: 10

    return 0;
}
```

vtable & vptr Concept

vtable (virtual table)

- A vtable is a table of function pointers used to support dynamic dispatch (runtime polymorphism) in C++.
- Each class with virtual functions has its own vtable, which contains pointers to the virtual functions defined in that class.
- When a derived class overrides a virtual function, the vtable for that derived class will contain a pointer to the overridden function instead of the base class's version.

vptr (virtual pointer)

- A vptr is a pointer that points to the vtable of a class.
- Each object of a class with virtual functions contains a vptr that points to its class's vtable.
- When a virtual function is called on an object, the vptr is used to look up the appropriate function pointer in the vtable, allowing the correct function to be invoked based on the actual object type at runtime.
- The vptr is initialized when an object of a class with virtual functions is created.

Pure Virtual Functions

- A pure virtual function is a virtual function that has no implementation in the base class and is declared by assigning 0 to it in the base class declaration.
- It is used to create abstract classes, which cannot be instantiated and serve as a blueprint for derived classes.

```
class Base {
public:
    virtual void display() = 0; // Pure virtual function
};
class Derived : public Base {
public:
    void display() override {
        // Derived class implementation of the pure virtual function
    }
};
```

Abstract Class

- An abstract class is a class that contains at least one pure virtual function.
- It cannot be instantiated directly and serves as a base class for derived classes that provide implementations for the pure virtual functions.
- When a virtual function is declared as pure virtual, the class becomes abstract, and any derived class must provide an implementation for that function to be instantiated.
- If a derived class does not implement all the pure virtual functions from its base class, it also becomes an abstract class and cannot be instantiated.

Interfaces

- An interface is a programming construct that defines a contract for classes to implement, specifying a set of methods that must be provided by any class that implements the interface.
- In C++, interfaces can be implemented using abstract classes with pure virtual functions.
- Interfaces are used to achieve multiple inheritance-like behavior in C++.

```
class MyInterface {
public:
    virtual void method1() = 0;
```

```
    virtual void method2() = 0;
};
```

Interface-style Thinking

- Interface-style thinking is a design approach that emphasizes defining clear contracts (interfaces) for classes to implement, promoting loose coupling and flexibility in software design.
- It encourages developers to focus on what a class should do (its behavior) rather than how it should do it (its implementation), allowing for easier maintenance, testing, and extensibility of code.
- This approach helps in creating more maintainable and scalable code by ensuring that classes adhere to well-defined contracts.

Introduction to Templates

- Templates in C++ are a powerful feature that allows developers to write generic and reusable code.
- They enable the creation of functions and classes that can operate with different data types without the need for code duplication.
- Templates provide a way to define algorithms and data structures that can work with any data type, making the code more flexible and maintainable.

Generic Programming Concept

- Generic programming is a programming paradigm that focuses on designing algorithms and data structures in a way that allows them to work with any data type.
- It promotes code reusability, type safety, and flexibility by allowing developers to write algorithms and data structures that can be used with different types without the need for code duplication.
- Generic programming is often achieved through the use of templates in C++, enabling developers to create functions and classes that can operate on a wide range of data types while maintaining type safety and efficiency.

Function Templates

- Function templates are a feature in C++ that allows developers to create generic functions that can operate on different data types without the need for code duplication.
- They are defined using the `template` keyword followed by a template parameter list, which specifies the types that the function can work with.
- When a function template is called with specific types, the compiler generates a specialized version of the function for those types, allowing the same function to be used with different data types.

```
template <typename T>
T add(T a, T b) {
    return a + b;
}

template <typename T, typename U>
T multiply(T a, U b) {
    return a * b;
```

```
}
```

Class Templates

- Class templates are a feature in C++ that allows developers to create generic classes that can operate on different data types without the need for code duplication.
- They are defined using the **template** keyword followed by a template parameter list, which specifies the types that the class can work with.
- When a class template is instantiated with specific types, the compiler generates a specialized version of the class for those types, allowing the same class to be used with different data types.

template : is a keyword used to define templates in C++. It allows developers to create generic functions and classes that can work with different data types without code duplication.

typename : is a keyword used in template parameter lists to specify that a parameter represents a type. It indicates that the parameter can be replaced with any valid data type when the template is instantiated.

T : is a commonly used placeholder name for a template parameter representing a type. It can be replaced with any valid data type when the template is instantiated.

```
template <typename T>
class Array {
private:
    T data;
public:
    Array(T value) : data(value) {}
    T getData() const { return data; }
};
```

Introduction to C++ Standard Template Library

- The C++ Standard Template Library (STL) is a powerful library that provides a collection of generic classes and functions for common data structures and algorithms.
- It is designed to work seamlessly with C++ templates, allowing developers to create efficient and reusable code for various data structures and algorithms without the need for manual implementation.

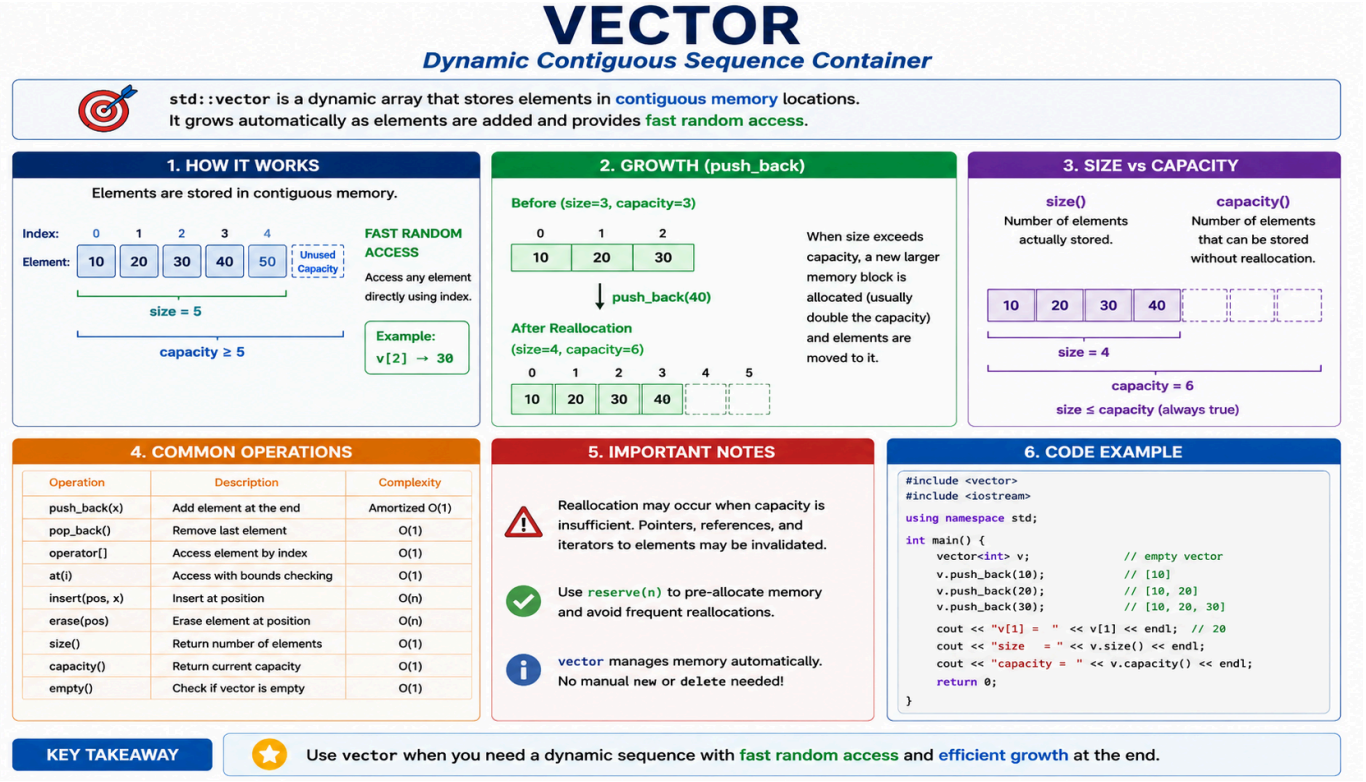
STL Containers

- The C++ Standard Template Library (STL) provides a variety of container classes that are used to store and manage collections of data.
- These containers are designed to be efficient, flexible, and easy to use, allowing developers to focus on solving problems rather than implementing data structures from scratch.

vector

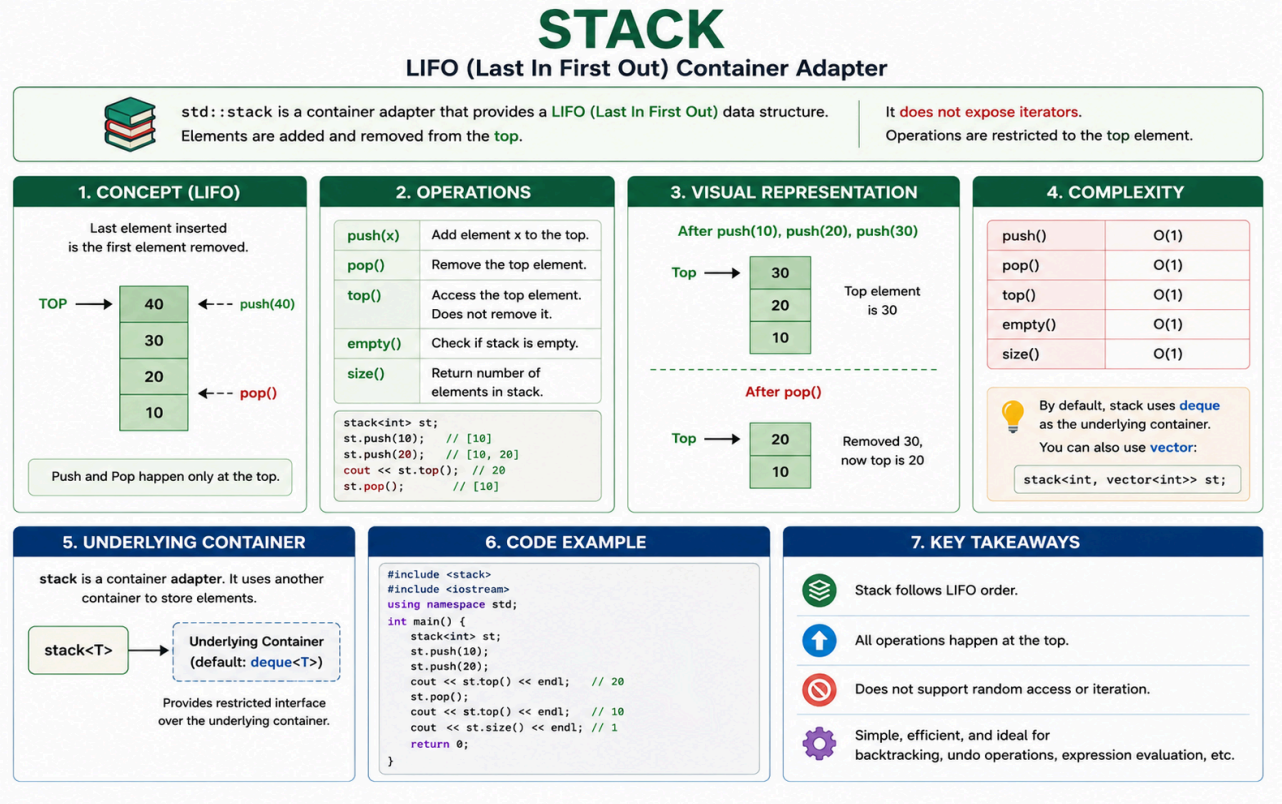
- A vector is a dynamic array that can grow and shrink in size as needed.

- It provides fast random access to elements and efficient insertion and deletion at the end of the container.
- Vectors are implemented as contiguous memory blocks, which allows for efficient memory usage and cache performance.



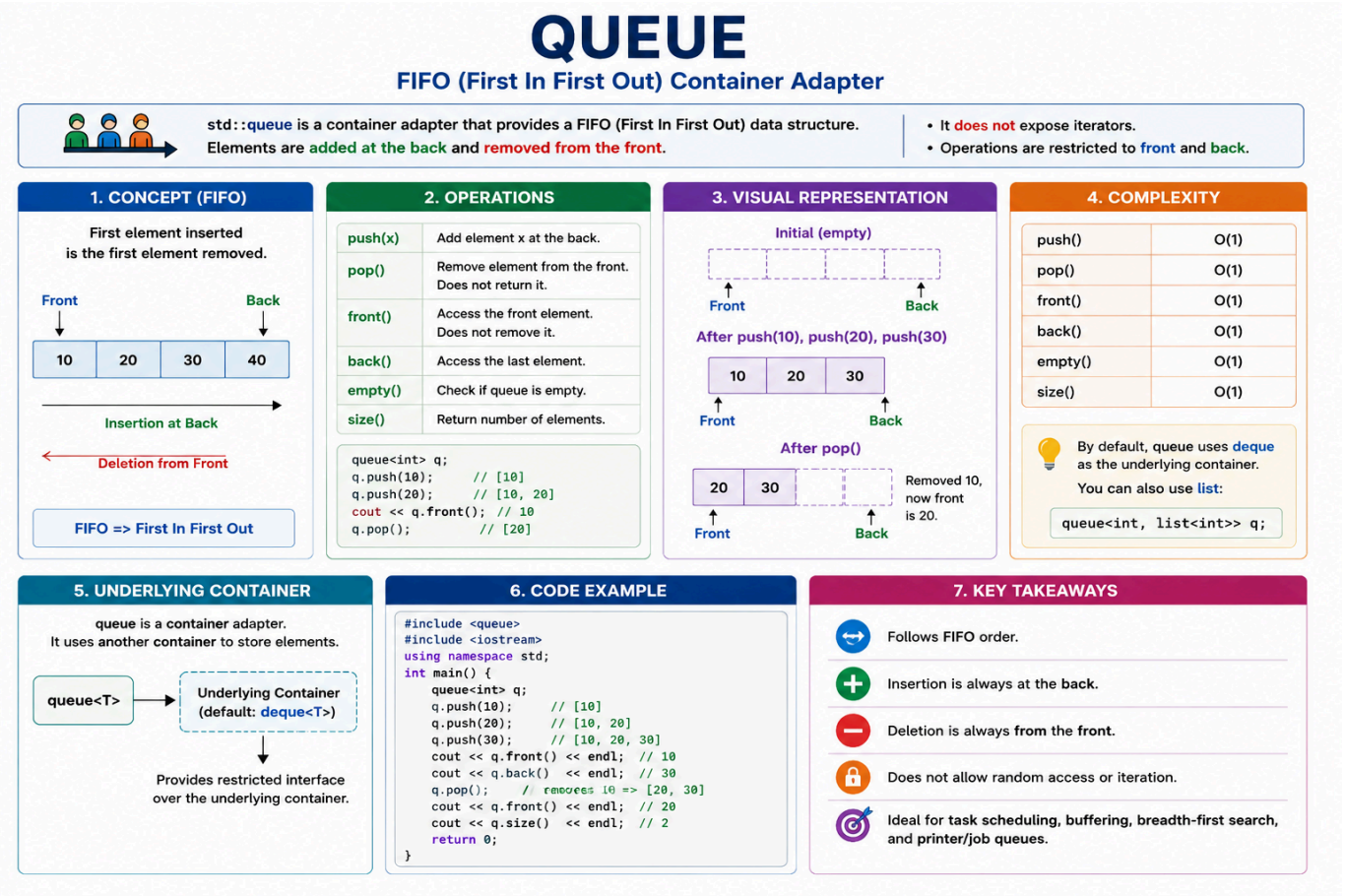
stack

- A stack is a container that follows the Last-In-First-Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed.
- It provides operations for adding (pushing) and removing (popping) elements from the top of the stack, as well as accessing the top element without removing it.
- Stacks are commonly used for tasks such as expression evaluation, backtracking algorithms, and managing function calls in programming languages.



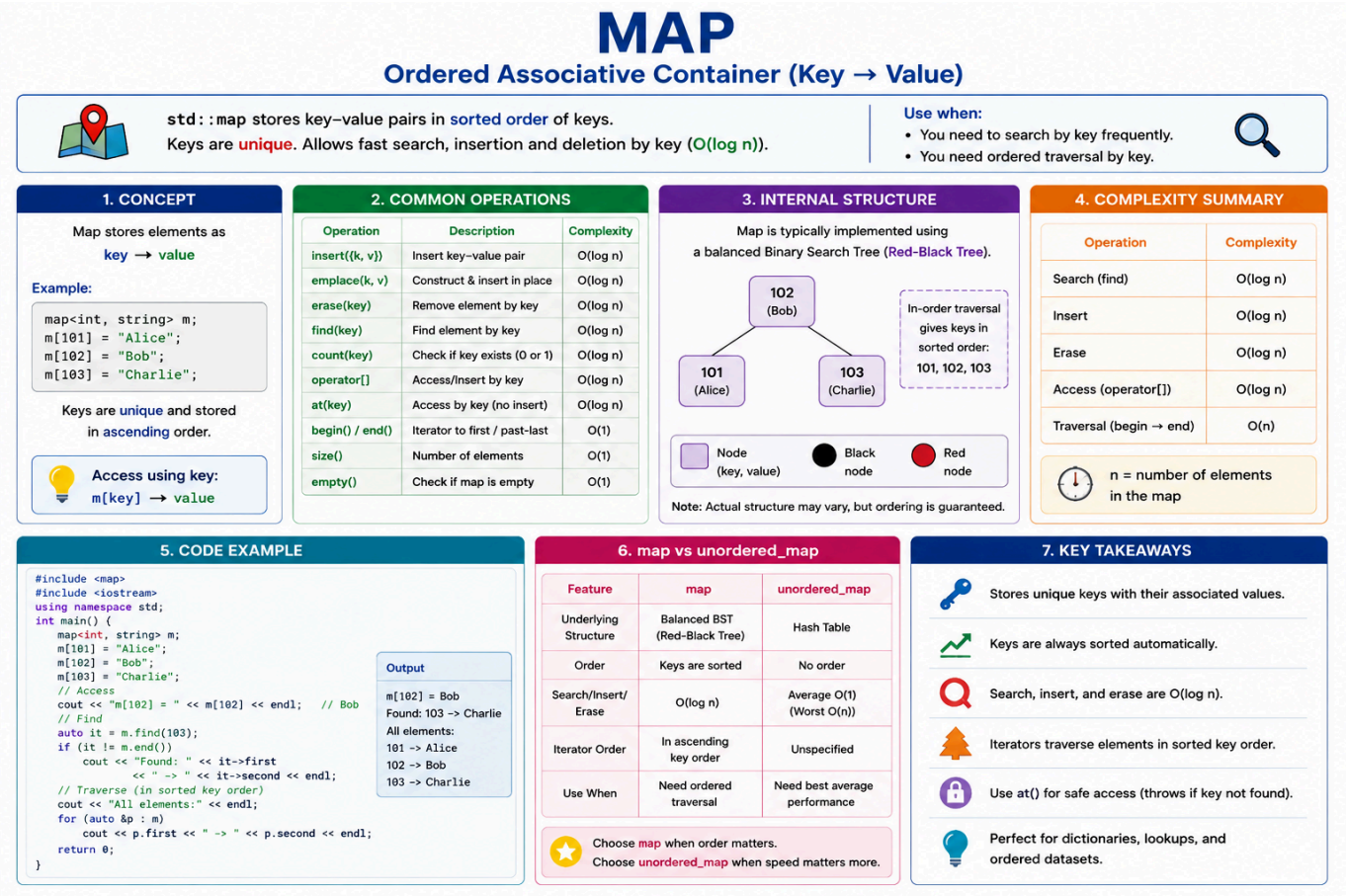
queue

- A queue is a container that follows the First-In-First-Out (FIFO) principle, meaning that the first element added to the queue is the first one to be removed.
- It provides operations for adding (enqueueing) and removing (dequeueing) elements from the front and back of the queue, as well as accessing the front and back elements without removing them.
- Queues are commonly used for tasks such as scheduling, buffering, and managing resources in concurrent systems.



map

- A map is an associative container that stores key-value pairs, where each key is unique and maps to a specific value.
- It provides fast lookup, insertion, and deletion of elements based on their keys, using a balanced binary search tree (typically a red-black tree) for efficient performance.
- Maps are commonly used for tasks such as implementing dictionaries, symbol tables, and caching mechanisms.



Basic STL Usage

- To use the STL, you need to include the appropriate header files for the containers and algorithms you want to use.
- For example, to use vectors, you would include the header file.
- You can create instances of STL containers and use their member functions to perform various operations, such as adding, removing, and accessing elements.
- STL algorithms can be used with STL containers to perform common operations, such as sorting, searching, and transforming data.
- These algorithms are designed to work with iterators, which provide a uniform way to access elements in different types of containers.

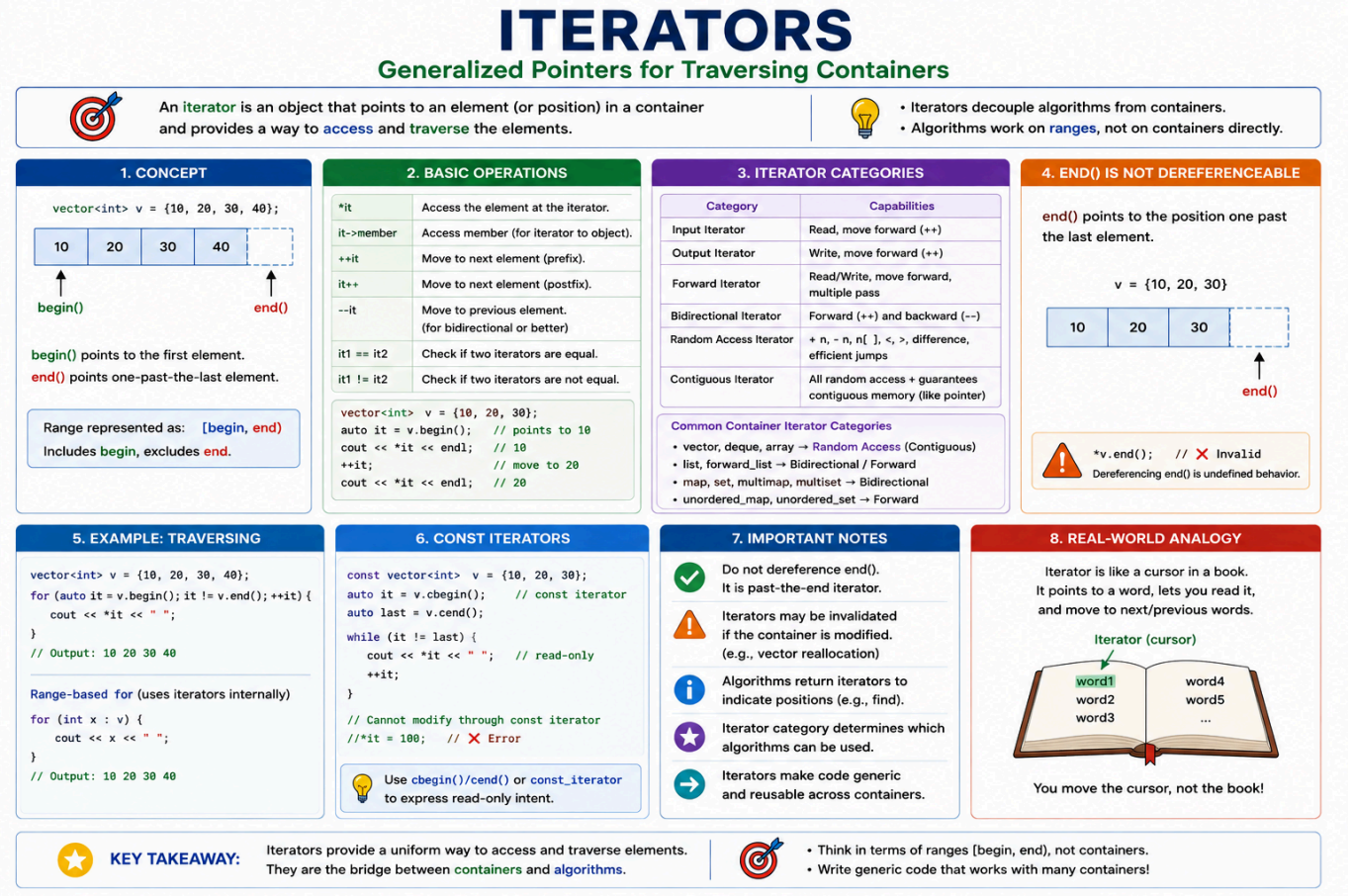
Iterators — Introduction

- Iterators are objects that provide a way to access and traverse the elements of a container in a sequential manner.
- They act as a bridge between containers and algorithms, allowing algorithms to work with different types of containers in a uniform way.
- Iterators provide a set of operations for moving through the elements of a container, such as incrementing, decrementing, and dereferencing to access the value of the element they point to.

Syntax:

```
auto it = container.begin(); // Create an iterator pointing to the first element
of the container
```

```
++it; // Move the iterator to the next element
--it; // Move the iterator to the previous element
*it; // Dereference the iterator to access the value of the element it points to
container.end(); // Get an iterator pointing to one past the last element of the
container
```



Algorithms — Basic Usage

- STL algorithms are a collection of functions that perform common operations on containers, such as sorting, searching, and transforming data.
- They are designed to work with iterators, allowing them to operate on different types of containers in a uniform way.
- To use STL algorithms, you need to include the appropriate header files, such as for sorting and searching algorithms.
- You can call STL algorithms by passing iterators that define the range of elements to operate on, along with any additional parameters required by the algorithm.

Algorithm Syntax:

```
#include <algorithm>

int main() {
    std::vector<int> vec = {5, 2, 9, 1, 5, 6};

    // Sort the vector in ascending order
```

```cpp
    std::sort(vec.begin(), vec.end());

    // Find an element in the vector
    auto it = std::find(vec.begin(), vec.end(), 5);
    if (it != vec.end()) {
        // Element found
    }

    return 0;
}
```

ALGORITHMS

Generic Functions That Operate on Ranges

The <algorithm> header provides a rich set of functions that work on ranges defined by iterators: [begin, end).

Algorithms are container-agnostic! They work with any container that provides the required iterator type.

1. WHAT IS AN ALGORITHM?

An algorithm is a function template that operates on a range of elements, specified by two iterators:

```
algorithm(begin, end, ...);
```

The range includes elements from begin and excludes end.

Example:

```
vector<int> v = {4, 1, 7, 2, 5};
sort(v.begin(), v.end());    // sort the range
                             // [v.begin(), v.end())
```

Before:	4	1	7	2	5
After sort():	1	2	4	5	7

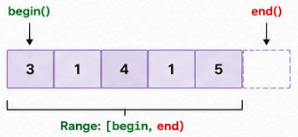
2. COMMON ALGORITHMS

Algorithm	Purpose	Returns	Modifies Range	Complexity
find(first, last, value)	Finds first occurrence of value	Iterator to element or last	No	O(n)
count(first, last, value)	Counts occurrences of value	Count (size_type)	No	O(n)
sort(first, last)	Sorts elements in ascending order	void	Yes	O(n log n)
reverse(first, last)	Reverses the order of elements	void	Yes	O(n)
for_each(first, last, func)	Applies func to each element	func	Depends on func	O(n)
max_element(first, last)	Finds largest element	Iterator to max element	No	O(n)
min_element(first, last)	Finds smallest element	Iterator to min element	No	O(n)
binary_search(first, last, value)	Checks if value exists in sorted range	bool	No	O(log n)

3. HOW THEY WORK

Algorithms work on iterator ranges.
They don't know or care about the container.

Example with vector<int> v = {3, 1, 4, 1, 5};



Includes 5 elements
(end is one-past-last)

4. EXAMPLES IN ACTION

find()
Find the value 4
auto it = find(v.begin(), v.end(), 4);

3	1	4	1	5
		↑ it		

count()
Count how many 1's
int c = count(v.begin(), v.end(), 1);
c = 2

sort()
Sort in ascending order
sort(v.begin(), v.end());

Before:	3	1	4	1	5
After :	1	1	3	4	5

reverse()
Reverse the range
reverse(v.begin(), v.end());

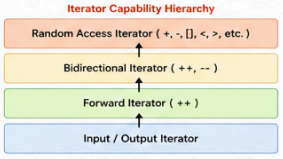
Before:	1	1	3	4	5
After :	5	4	3	1	1

max_element()
Find the maximum element
auto it = max_element(v.begin(), v.end());
Max = 5

5. REQUIREMENTS (ITERATOR CATEGORIES)

Algorithm	Minimum Required Iterator
find, count, for_each, max_element, min_element	Input Iterator
reverse	Bidirectional Iterator
sort, binary_search	Random Access Iterator

Iterator Capability Hierarchy



6. CODE EXAMPLE

```cpp
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;
int main() {
   vector<int> v = {4, 2, 5, 1, 3, 2};

   sort(v.begin(), v.end());
   cout << "Sorted: ";
   for (int x : v) cout << x << ' ';
   cout << endl;   // 1 2 2 3 4 5

   auto it = find(v.begin(), v.end(), 3);
   if (it != v.end())
      cout << "3 found at index " << (it - v.begin()) << endl;

   cout << "Count of 2 = " << count(v.begin(), v.end(), 2) << endl;

   reverse(v.begin(), v.end());
   cout << "Reversed: ";
   for (int x : v) cout << x << ' ';
   return 0;
}
```

KEY TAKEAWAY: Algorithms + Iterators = Powerful, Reusable, and Efficient!

Write once, use with many containers.
Think in terms of ranges, not containers.